

NS-3 Tutorial#1

Index

- Introduction to NS-3 - Installation
- First example- Scratch-simulator.cc
- Second example- First.cc
- Different ways to trace the results

Installation of NS-3

- <https://www.nsnam.org/docs/release/3.42/installation/html/quick-start.html>

```
$ wget https://www.nsnam.org/releases/ns-allinone-3.42.tar.bz2
```

```
$ tar xvj ns-allinone-3.42.tar.bz2
```

```
$ cd ns-allinone-3.42/ns-3.42
```

```
$ ./ns3 configure --enable-examples --enable-tests
```

```
$ ./ns3 build
```

Once build successfully. Test the software.

```
$ ./test.py
```

Once test is completed. Run the script.

```
$ ./ns3 run scratch/scratch-simulator.cc
```

It should print **Scratch simulator** in the terminal.

Introduction to NS-3



Some important folders:

- Scratch
- Src
- Doc
- Examples

Few terminology

1. Node
2. Application
3. Channel
4. Net Device
5. Topology helpers

First script: Scratch-Simulator.cc

```
17 #include "ns3/core-module.h"
```

Include the files

```
18
```

```
19 using namespace ns3;
```

The ns-3 project is implemented in a C++ namespace called ns3. This groups all ns-3-related declarations in a scope outside the global namespace

```
20
```

```
21 NS_LOG_COMPONENT_DEFINE ("ScratchSimulator");
```

```
22
```

```
23 int
```

```
24 main (int argc, char *argv[])
```

```
25 {
```

```
26     NS_LOG_UNCOND ("Scratch Simulator");
```

```
27
```

```
28     Simulator::Run ();
```

```
29     Simulator::Destroy ();
```

```
30 }
```

A logging component called ScratchSimulator that allows you to enable and disable console message logging by reference to the name.

Log the associated message unconditionally

run the simulation using the global function Simulator::Run.

All that remains is to clean up

Second Example: First.cc

```
16
17 #include "ns3/core-module.h"
18 #include "ns3/network-module.h"
19 #include "ns3/internet-module.h"
20 #include "ns3/point-to-point-module.h"
21 #include "ns3/applications-module.h"
```

```
22
23 // Default Network Topology
24 //
25 //      10.1.1.0
26 // n0 ----- n1
27 //      point-to-point
28 //
```

```
29
30 using namespace ns3;
```

```
31
32 NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");
```

```
33
34 int
35 main (int argc, char *argv[])
36 {
37     CommandLine cmd (__FILE__);
38     cmd.Parse (argc, argv);
```

```
39
40     Time::SetResolution (Time::NS);
41     LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);
42     LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);
```

```
43
44     NodeContainer nodes;
45     nodes.Create (2);
```

```
46
47     PointToPointHelper pointToPoint;
48     pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
49     pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
```

```
50
51     NetDeviceContainer devices;
52     devices = pointToPoint.Install (nodes);
53
```

Sets time resolution to one nsec. i.e., the smallest time value that can be represented.

The next two lines of the script are used to enable two logging components that are built into the Echo Client and Echo Server applications

create the ns-3 Node objects and define the value to 2 nodes.

Physical layer:
constructing a point to point link

Create ns-3 device objects and add P2P channel between the nodes

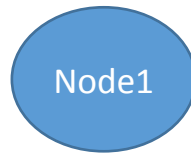
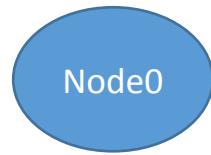
The InternetStackHelper is a topology helper that is to internet stacks what the PointToPointHelper is to point-to-point net devices.

```
51 NetDeviceContainer devices;  
52 devices = pointToPoint.Install (nodes);  
53  
54 InternetStackHelper stack;  
55 stack.Install (nodes);  
56  
57 Ipv4AddressHelper address;  
58 address.SetBase ("10.1.1.0", "255.255.255.0");  
59  
60 Ipv4InterfaceContainer interfaces = address.Assign (devices);  
61  
62 UdpEchoServerHelper echoServer (9);  
63  
64 ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));  
65 serverApps.Start (Seconds (1.0));  
66 serverApps.Stop (Seconds (10.0));  
67  
68 UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);  
69 echoClient.SetAttribute ("MaxPackets", UintegerValue (1));  
70 echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));  
71 echoClient.SetAttribute ("PacketSize", UintegerValue (1024));  
72  
73 ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));  
74 clientApps.Start (Seconds (2.0));  
75 clientApps.Stop (Seconds (10.0));  
76  
77 Simulator::Run ();  
78 Simulator::Destroy ();  
79 return 0;  
80 }
```

associate the devices with IP addresses

set up a UDP echo
server application on one
of the nodes at port no. 9

set up a UDP echo client
application on one of the
nodes 1

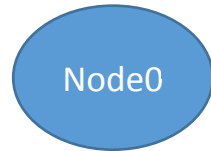


```
NodeContainer nodes;  
nodes.Create (2);
```

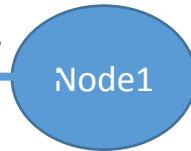
P2P: 5 Mbps, 2ms

```
PointToPointHelper pointToPoint;  
pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"))  
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
```

Physical
layer

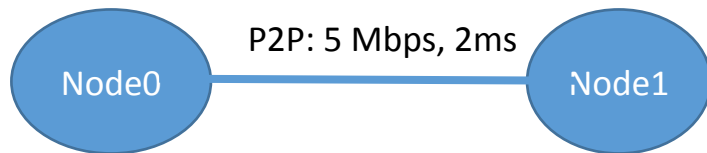


P2P: 5 Mbps, 2ms



```
NetDeviceContainer devices;  
devices = pointToPoint.Install (nodes);
```

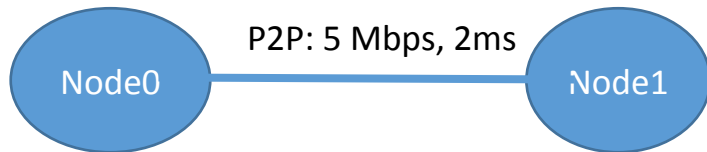
Internet Stack



Internet Stack

```
InternetStackHelper stack;  
stack.Install (nodes);
```

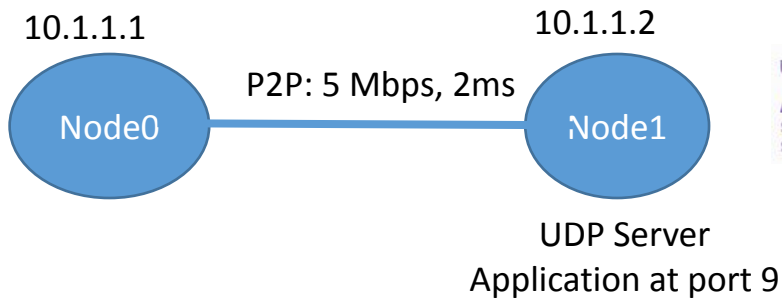
10.1.1.1



10.1.1.2

```
Ipv4AddressHelper address;  
address.SetBase ("10.1.1.0", "255.255.255.0");  
Ipv4InterfaceContainer interfaces = address.Assign (devices);
```

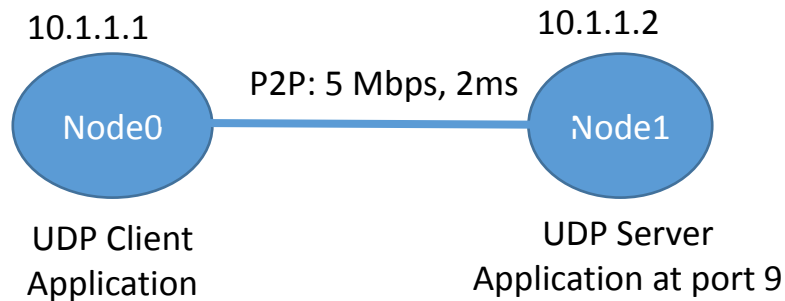
Internet
stack
helper



```
UdpEchoServerHelper echoServer (9);
```

```
ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));  
serverApps.Start (Seconds (1.0));  
serverApps.Stop (Seconds (10.0));
```

Application
layer



```
UdpEchoClientHelper echoClient (interfaces.GetAddress (1), 9);  
echoClient.SetAttribute ("MaxPackets", UIntegerValue (1));  
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));  
echoClient.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps = echoClient.Install (nodes.Get (0));  
clientApps.Start (Seconds (2.0));  
clientApps.Stop (Seconds (10.0));
```

Result

```
[2701/2801] Linking build/serenity/first
Waf: Leaving directory `/home/akshita/ns-allinone-3.30.1/ns-allinone-3.30.1/ns-3.30.1/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (3.532s)
At time 2s client sent 1024 bytes to 10.1.1.2 port 9
At time 2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time 2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time 2.00737s client received 1024 bytes from 10.1.1.2 port 9
akshita@akshita-HP-Laptop:~/ns-allinone-3.30.1/ns-allinone-3.30.1/ns-3.30.1$
```

Quick exercises:

- Change the delay of P2P link from 2 msec to 5 msec?

At time 2s client sent 1024 bytes to 10.1.1.2 port 9

At time 2.00669s server received 1024 bytes from 10.1.1.1 port 49153

At time 2.00669s server sent 1024 bytes to 10.1.1.1 port 49153

At time 2.01337s client received 1024 bytes from 10.1.1.2 port 9

- Change the data rate of the P2P link from 5 Mbps to 10 Mbps?

At time 2s client sent 1024 bytes to 10.1.1.2 port 9

At time 2.00284s server received 1024 bytes from 10.1.1.1 port 49153

At time 2.00284s server sent 1024 bytes to 10.1.1.1 port 49153

At time 2.00569s client received 1024 bytes from 10.1.1.2 port 9

- Change the data rate of the P2P link from 5 Mbps to 10 Mbps, and increase the packet size to 2048?

At time 2s client sent 2048 bytes to 10.1.1.2 port 9

At time 2.00536s server received 2048 bytes from 10.1.1.1 port 49153

At time 2.00536s server sent 2048 bytes to 10.1.1.1 port 49153

At time 2.01072s client received 2048 bytes from 10.1.1.2 port 9

- Change the server port number from 9 to 21?

At time 2s client sent 1024 bytes to 10.1.1.2 port 21

At time 2.00369s server received 1024 bytes from 10.1.1.1 port 49153

At time 2.00369s server sent 1024 bytes to 10.1.1.1 port 49153

At time 2.00737s client received 1024 bytes from 10.1.1.2 port 21